GUIA TÉCNICO EP / DISCIPLINA 04

NoSQL e sistemas distribuídos

Consistência, disponibilidade, particionamento e decisões de arquitetura

MAPA DE ESTUDO

01 Modelos NoSQL e padrões de acesso

02 Consistência e isolamento

03 Disponibilidade e falhas de rede

04 CAP, consenso e replicação

05 Escolha de arquitetura por requisitos

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

06 | NoSQL e sistemas distribuídos

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Consistência e particionamento

Não equipare NoSQL à ausência de esquema ou transações. O teorema CAP caracteriza escolhas sob partição de rede; não é uma tabela simples de desempenho. Replicação, particionamento e modelos de consistência têm impactos distintos.

Aplicação, ressalvas e limites

Para pagamentos, invariantes fortes podem justificar rejeitar temporariamente operações. Para contadores de visualização, atualização eventual pode ser aceitável se o produto tolerar defasagem.

Classes de bancos NoSQL

Documentos, chave-valor, wide-column e grafos resolvem problemas diferentes. A expressão “NoSQL” não significa automaticamente ausência de transação nem esquema.

Exemplo do notebook — execução local

```
tipos=pd.DataFrame({'tipo':['Documento','Chave-valor','Wide-column','Grafo']
    ,'exemplo':['MongoDB','Redis','Cassandra','Neo4j'],'consulta_tipica':['f
    iltro de documento','busca por chave','consulta por partição','travessia
    s']});display(tipos)
```

CAP sob partição de rede

Em presença de uma partição de rede, a aplicação enfrenta trade-off entre disponibilidade das requisições e consistência forte (linearizabilidade). Não é uma escolha universal de quaisquer duas letras em operação normal.

Exemplo do notebook — execução local

```
print('Pergunta de projeto: durante uma partição, podemos rejeitar operações
     para preservar consistência?')
```

Consistência eventual e replicação

Replicar dados não garante resposta imediatamente atualizada em todos os nós. Consistência de transação ACID difere da propriedade de consistência do CAP.

Exemplo do notebook — execução local

```
cenarios=pd.DataFrame({'cenario':['Saldo financeiro','Curtidas em rede socia
    l','Log de sensores'],'preocupacao':['invariantes fortes','atualização p
    ossivelmente assíncrona','throughput de escrita']});display(cenarios)
```

Exercício resolvido - decisão arquitetural

Para um sistema de pontos acumulados, tolerar temporariamente uma leitura desatualizada pode ser aceitável; para impedir gasto duplo, exige-se um desenho com invariantes fortes. Justifique por operação, não apenas por rótulo do banco.

--- ### Aprofundamento v2 - CAP não é uma régua de desempenho

O teorema CAP é sobre garantias sob partição de rede: não afirma que sistemas normais escolhem apenas duas de três propriedades o tempo todo. Diferencie consistência CAP (observação de dados) de consistência ACID (invariantes das transações). Considere latência, durabilidade, quórum e recuperação de falhas nas decisões.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
politicas = {
    'pagamento': {'integridade': 'forte', 'tolerancia_desatualizacao': False
    },
    'contador_visualizacoes': {'integridade': 'eventual_admissivel', 'tolera
    ncia_desatualizacao': True}
}
assert politicas['pagamento']['tolerancia_desatualizacao'] is False
print(politicas)
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Consistência como requisito

Uma decisão de arquitetura começa por requisitos mensuráveis. Qual o atraso aceitável em um contador? Qual a consequência de mostrar saldo antigo? Qual o tempo de recuperação desejado após falha? Respostas diferentes produzem escolhas diferentes. Não use rótulos vagos como “mais escalável” sem quantificar carga, latência, volume e padrões de acesso.

CAP sob partição

O teorema CAP trata de escolhas quando há partição de rede. Não diz que um sistema distribuído escolhe exatamente duas propriedades de uma lista fixa em condições normais, nem significa que o sistema consistente é sempre mais lento. A política de leitura/escrita, quorum, confirmação e resolução de conflitos precisa ser descrita no contexto do sistema concreto.

Replicação não é particionamento

Replicar cria cópias dos dados; particionar divide o universo em grupos. Um cluster pode empregar as duas técnicas, com diferentes custos operacionais. Maior redundância pode melhorar tolerância a falhas, mas não elimina necessidade de backups, autenticação, monitoramento e testes de recuperação.

Exercício de decisão

Para pagamentos, comentários de rede social e registros de sensores, proponha uma política de consistência, tolerância a indisponibilidade e estratégia de recuperação. Explique como sua escolha mudaria caso o sistema exigisse confirmação síncrona de cada gravação. Identifique ainda que evidências reais seriam necessárias para validar a proposta.

Checklist de interpretação

- Defina a pergunta, unidade de análise e pré-condições da consulta.
- Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.
- Execute os exemplos locais antes de habilitar serviços reais.
- Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.